

Rapport réunion 03-06-2026

Initialement pour se faire la main en implémentant une décomposition tensorielle dans le notebook démonstrateur de l'obtention de la solution d'une poutre en déformation axiale

(https://github.com/AlexandreDabySeesaram/Simplified_NeuROM_demos).

J'ai implémenté de la r-adaptivity (node relocation).

On pourra trouver le repo ici : https://github.com/Solalspc/Simplified_NeuROM_demos

Problème général

On veut maintenant chercher la solution sous la forme

$$u(x, E) = \sum_{i=1}^m u_i(x) \lambda_i(E)$$

comme ça on a le module d'Young en paramètre.

Donc l'énergie qu'on minimise est maintenant :

$$\begin{aligned} \mathcal{E} &= \int_{I_E} \int_{\Omega_{ref}} \left[\frac{1}{2} E \nabla u \cdot \nabla u - f u \right] J^u J^E dx dE \\ &= \int_{I_E} \int_{\Omega_{ref}} \left[\frac{1}{2} E \nabla(u_i \lambda_i) \cdot \nabla(u_j \lambda_j) - f u_k \lambda_k \right] J^u J^E dx dE \\ &= \left(\int_{I_E} \frac{1}{2} E \lambda_i \lambda_j J^E dE \right) \left(\int_{\Omega} \nabla u_i \nabla u_j J^u dx \right) \\ &\quad - \left(\int_{I_E} \lambda_k J^E dE \right) \left(\int_{\Omega} f u_k J^u dx \right) \\ &= \frac{1}{2} E_e \lambda_{ie} \lambda_{je} J_e^E \nabla u_{ix} \nabla u_{jx} J_x^u - \lambda_{ke} J_e^E f_x u_{kx} J_x^u \end{aligned}$$

Avec la discrétisation du domaine Ω et I_E qui fait que

$$\lambda_{ke} = \lambda_k(E_{gauss,e}) \text{ et } u_{kx} = u_k(x_{gauss,x})$$

et de même pour les déterminants E et f .

On peut donc effectuer le calcul de l'énergie avec quelques appels de `torch.einsum()` bien sentis.

R-adaptivity

Implémentation

Les coordonnées sont stockées dans `self.coordinates = nn.ParameterDict({'all' : ...})`.

En l'absence de r-adapt, on initialise avec le mesh donné en input (`nodes_u`).

Avec r-adapt, il faut bien que les noeuds restent ordonnés pour ne pas avoir d'éléments qui s'interpénètrent et il faut bien que le premier soit égal à $a = 0$ et le dernier à b .

Au lieu d'opérer par pénalisation dans la loss, on encode ces contraintes dans la logique du réseau.

Pour cela, on opère une reparamétrisation :

- `self.coordinates` deviennent des réels quelconques. Ils peuvent varier dans $\mathbb{R} : c_i$

- Puis on applique la fonction $\text{softplus}(x) = \ln(1 + e^x)$ qui transforme ce paramètre quelconque en un réel positif (adouci par le $\ln$). On a donc à ce moment un incrément d'un point à l'autre : δ_j
- On fait une somme cumulative pour retrouver les coordonnées physiques des points du mesh.

$$\tilde{x}_k = \sum_{i=1}^{k-1} \delta_i = \sum_{i=1}^{k-1} \text{softplus}(c_i)$$
- Puis on renormalise $x_k = a + (b - a) \cdot \frac{\tilde{x}_k}{\tilde{x}_n}$. De cette manière les contraintes de bords sont toujours respectées.

Si notre `self.coordinates = c_i` alors $x_k = \sum_{i=1}^{k-1} \text{softplus}(c_i)$.

On encode cette logique dans une fonction `self.get_coordinates()` qui va remplacer `self.coordinates['all']` dans le code.

En l'absence de r-adapt, cette fonction est juste l'identité i.e. `self.coordinates` représente vraiment les coordonnées physiques des points du maillage.

Pour initialiser correctement le mesh, on fait le travail inverse.

Étant donné le mesh en input qui est celui avec lequel on initialise, on calcule les incréments entre les points puis on compose par la fonction inverse du softplus. Cela donne les "coordonnées généralisées" du mesh en input.

On gère quelles variables sont entraînées durant le processus d'ajout de mode. On initialise donc toutes les variables avec `requires_grad = False` puis on les passe à `True` selon les options.

De cette manière, on peut exactement contrôler quels paramètres du modèle sont optimisés ou non à chaque étape de l'enrichissement modal.

Le problème des meshes différents : intersection de maillage

On va optimiser la position des noeuds. L'idée étant que plus on monte en ordre de mode, plus on va corriger des erreurs des modes précédents.

Il serait donc utile de pouvoir faire vivre chaque mode sur un mesh différent.

Il faut alors faire attention dans les intégrales nécessaires au calcul de l'énergie.

- Le terme volumique $\int_{\Omega} f u_k J^u dx$ fait intervenir un mode à la fois, il n'y a donc rien de particulier à faire, les intégrales vont être calculées sur des meshes différents aux points de Gauss associés.
- Le terme de gradient $\int_{\Omega} \nabla u_i \nabla u_j J^u dx$ fait interagir deux modes qui ne sont pas discrétisés sur la même grille. Il faut donc les projeter sur une grille commune. Pour cela, on fait l'intersection des maillages et on calcule les points de Gauss et déterminants associés :

```
def intersect_grids(i_grid, j_grid, tol=1e-10):
    # Force 1D regardless of input shape
    i_grid = i_grid.reshape(-1)
    j_grid = j_grid.reshape(-1)
    1. Union + sort
    merged, _ = torch.sort(torch.cat([i_grid, j_grid]))

    # 2. Remove near-duplicates
    gaps = torch.diff(merged)
    keep = torch.cat([torch.tensor([True], device=merged.device), gaps > tol])
    nodes = merged[keep]

    # 3. Element quantities
    h = torch.diff(nodes)
    gauss_pts = nodes[:-1] + 0.5 * h
```

```
det_J = 0.5 * h
return det_J, gauss_pts
```

On passe ensuite le modèle en mode `.eval()`, on l'évalue aux points de Gauss (dont on suit le gradient en activant `requires_grad = True`) puis on utilise l'autograd de Pytorch avec `create_graph = True` pour calculer ces nouvelles dérivées.

Dans le calcul de l'énergie il faut alors faire attention au fait que les déterminants et points de Gauss dépendent maintenant du mode concerné :

$$\mathcal{E} = \frac{1}{2} E_e \lambda_{ie} \lambda_{je} J_e^E \nabla u_{ix'} \nabla u_{jx'} J_{ijx'}^u - \lambda_{ke} J_e^E f_x u_{kx} J_{kx}^u$$

Résultats

On va tester plusieurs cas, selon quels paramètres sont optimisés et à quel stade.

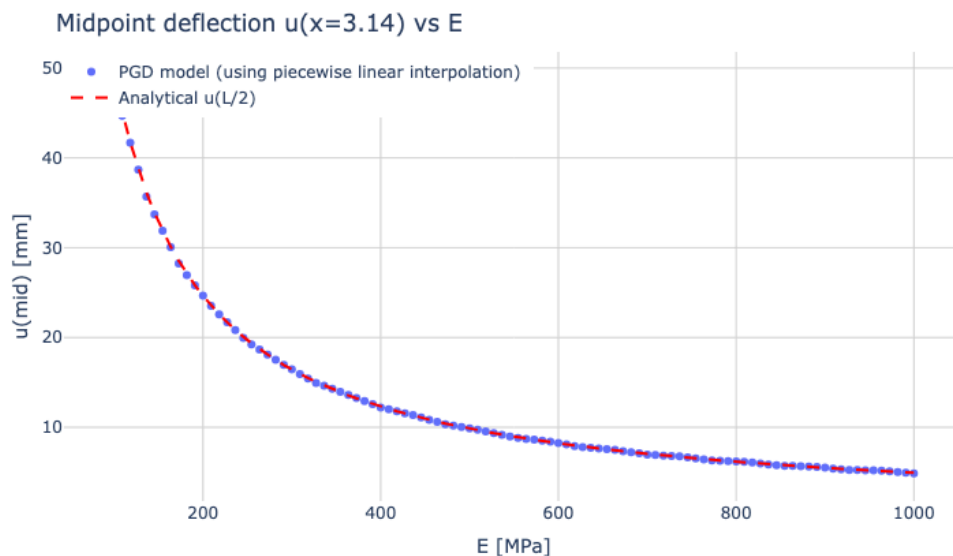
On prend comme grilles :

- grille uniforme de 100 à 1000 MPa de 25 noeuds pour la discrétisation en E
- grille uniforme de 0 à 2π de 25 noeuds pour la discrétisation en x puis dans la suite, pour la grille non-uniforme on prend une grille selon une fonction carrée renormalisée.

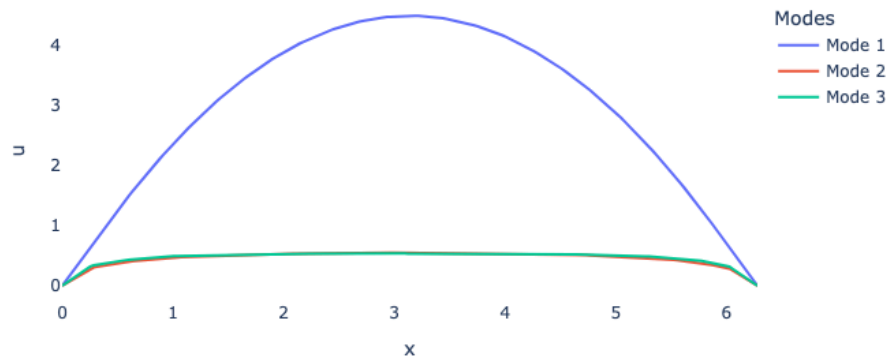
no r-adapt | nodal values per mode | meshes per mode

On va ensuite considérer ces essais-là comme référence pour de potentielles comparaisons.

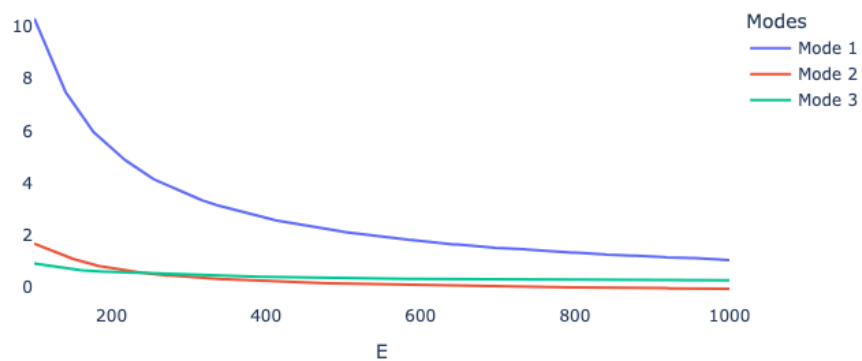
On converge très vite et de manière robuste (Sans efforts particulier de tuning du learning rate).



PGD spatial modes



PGD parametric modes



À partir de là, on prend une grille non-uniforme pour initialiser le mesh spatial.

Plus spécifiquement, on prend le carré renormalisé de la grille uniforme.

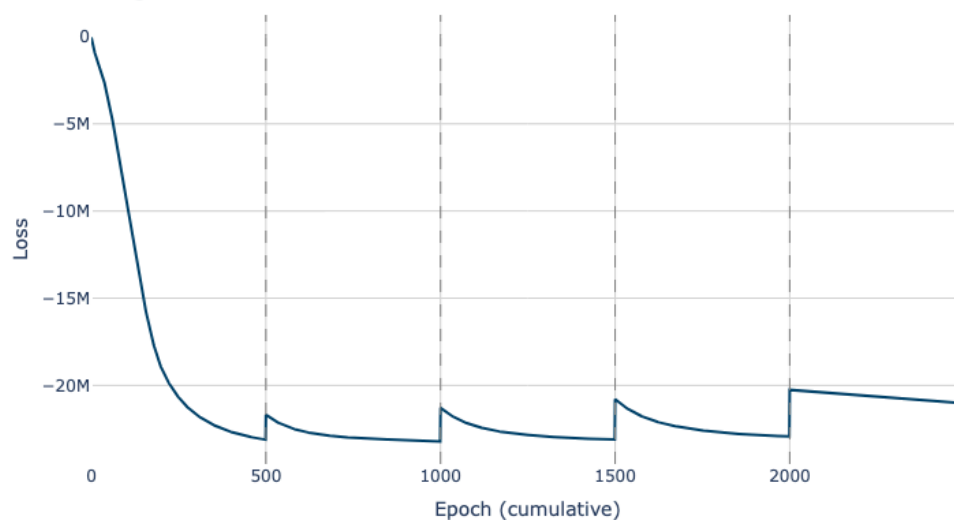
r-adapt | nodal values per mode | meshes per mode

On converge sans effort.

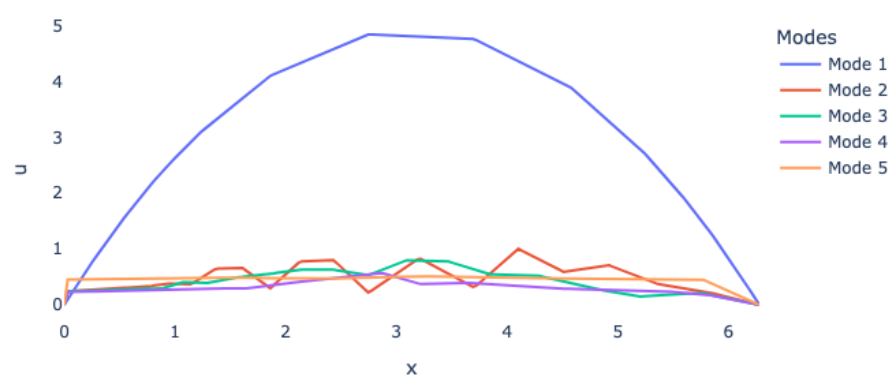
On voit que la grille du mode 1 s'adapte pour aller densifier plus proche des extrémités.

On ne réussit pas à reproduire la dépendance en E correctement dans la zone où c'est le plus difficile (basses valeurs de E).

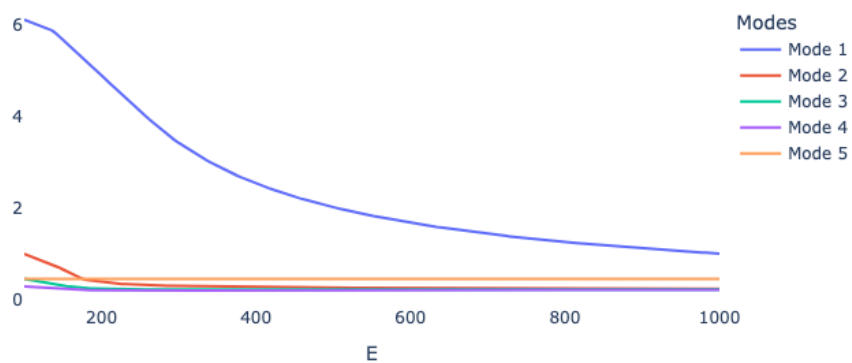
Training loss — all modes

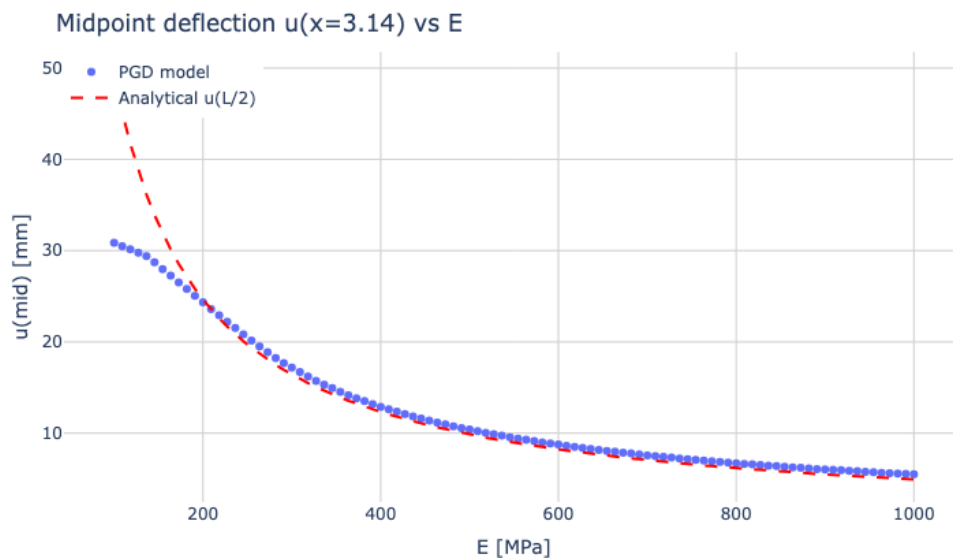
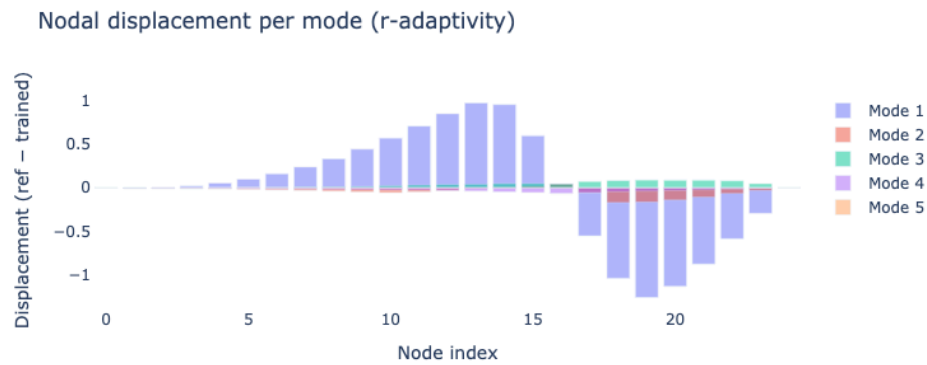


PGD spatial modes



PGD parametric modes





r-adapt | nodal values all at once | meshes per mode

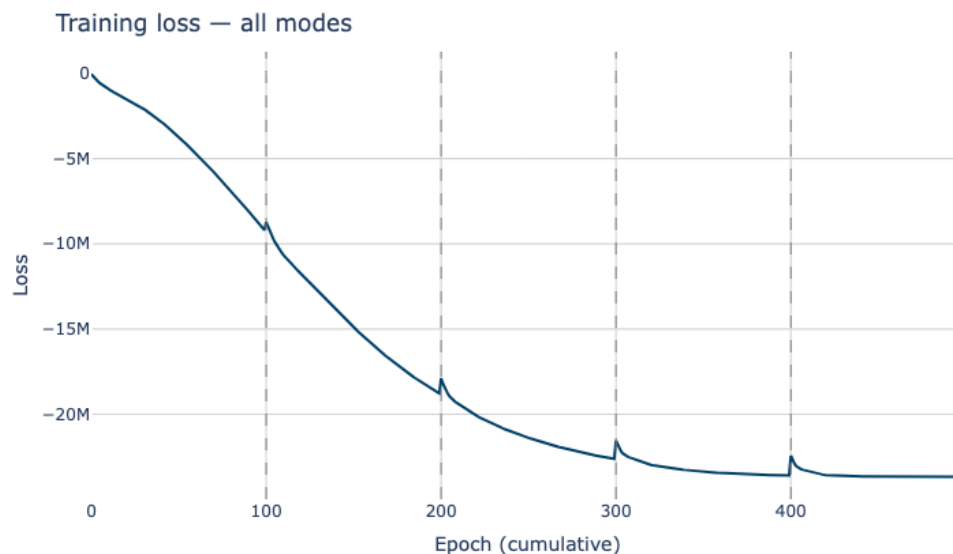
On revient au cas du notebook initial. Mais avec de la r-adaptivity.

On entraîne donc les valeurs nodales de tous les modes à la fois.

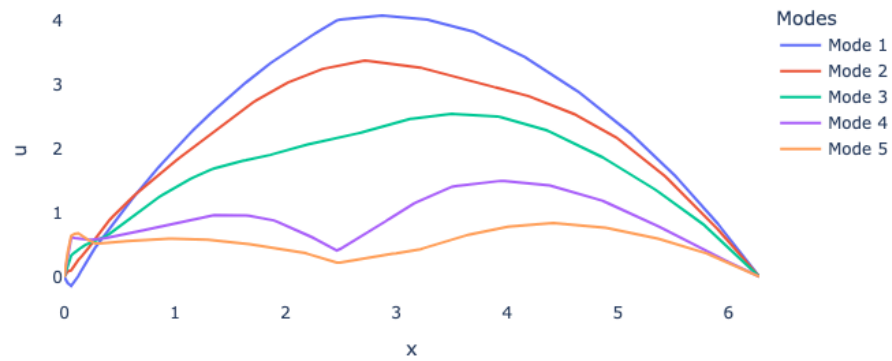
Mais on entraîne seulement le mesh du dernier mode ajouté.

Pour observer la correction d'un mode à l'autre je choisis de faire peu d'epochs à chaque ajout de mode.

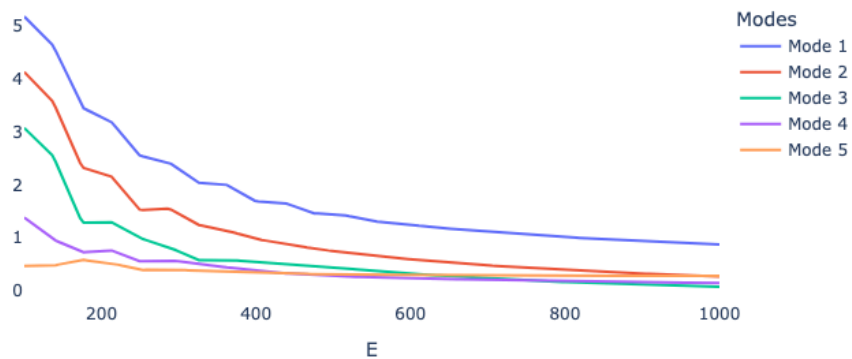
On a donc des modes assez peu convergés mais quand on en a suffisamment, on finit par avoir une solution convenable et la dépendance en E est bien reproduite.



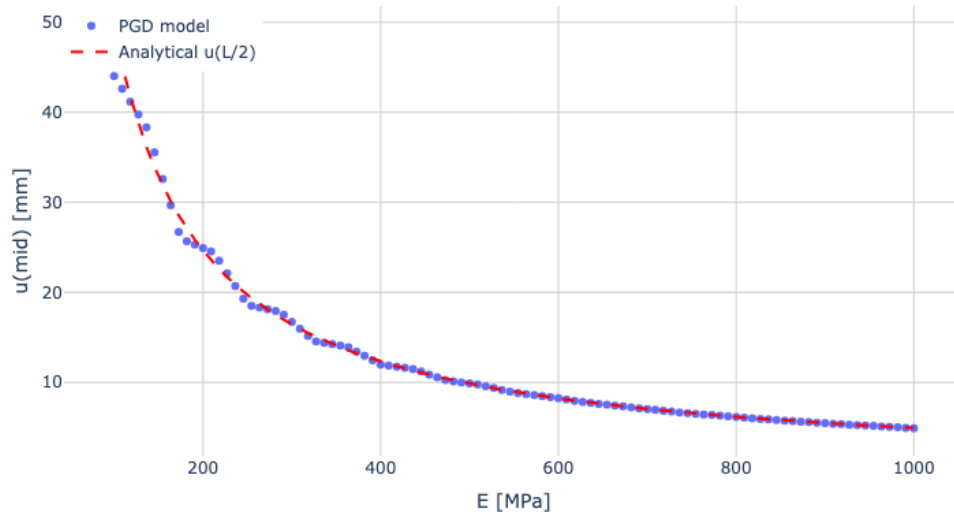
PGD spatial modes



PGD parametric modes



Midpoint deflection $u(x=3.14)$ vs E



r-adapt | nodal values per mode | meshes all at once

Maintenant on entraîne les coordonnées des meshes de tous les modes à la fois.

Mais on entraîne seulement les valeurs nodales du dernier mode ajouté.

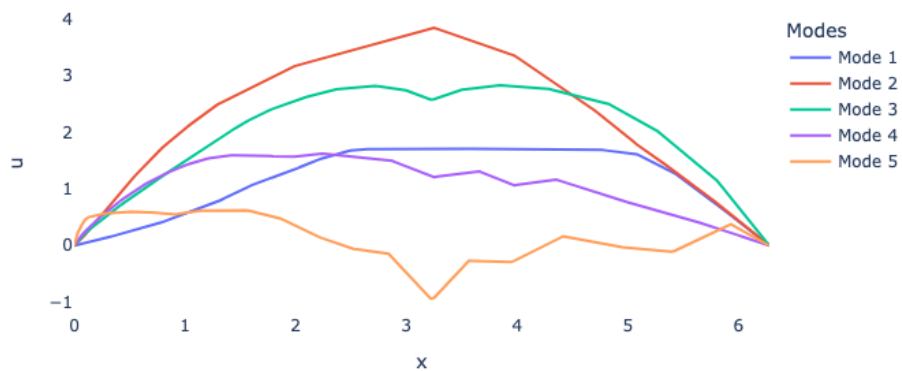
Remarque ADS :

Cela a-t-il un sens de bouger les coordonnées d'un noeud sans que sa valeur nodale ne change ? Le couple (valeur nodale, noeuds) a-t-il encore un sens ?

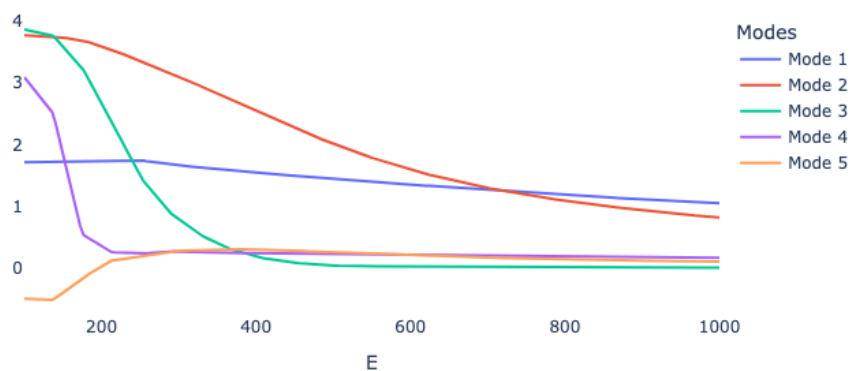
Le modèle réussit à approximer correctement la solution mais est-il encore interprétable de la même manière ?

Il faudrait peut-être voir si lors de l'ajout d'un nouveau mode, les coordonnées des noeuds des modes précédents évoluent dans la pratique ou bien si, malgré qu'on l'autorise, ils n'évoluent pas.

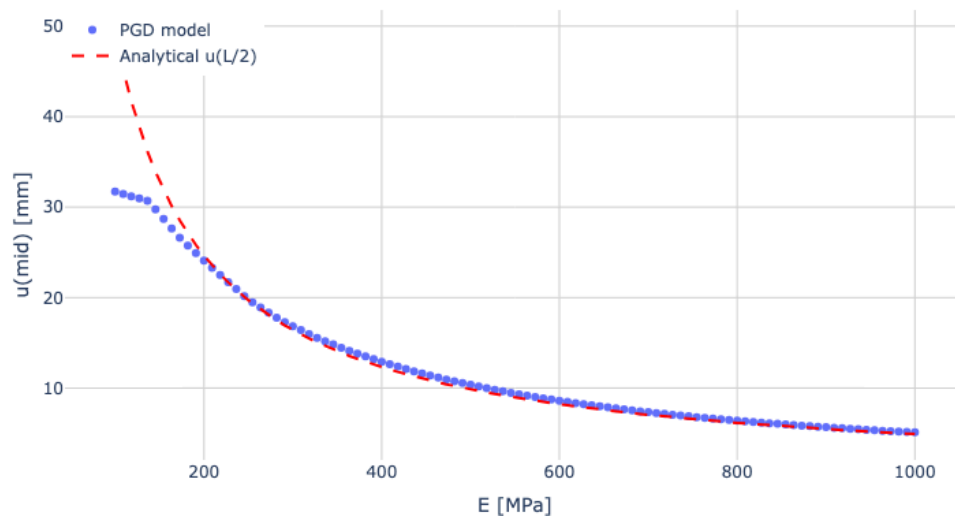
PGD spatial modes



PGD parametric modes



Midpoint deflection $u(x=3.14)$ vs E



r-adapt | nodal values all at once | meshes all at once

Maintenant on entraîne toujours les valeurs nodales de TOUS les modes et les coordonnées du mesh de TOUS les modes à la fois.

~~C'était le cas puis le lendemain, tout convergeait quel que soit le learning rate que je mettais. Quand ça ne convergeait pas, les conditions de bords n'étaient plus respectées par les modes, ce qui devrait être impossible par construction du réseau, il y avait peut-être un bug plus profond.~~

On a bien quelque chose qui converge de manière assez robuste, avec des learning rates de $1e-2$ ou même $1e-3$.

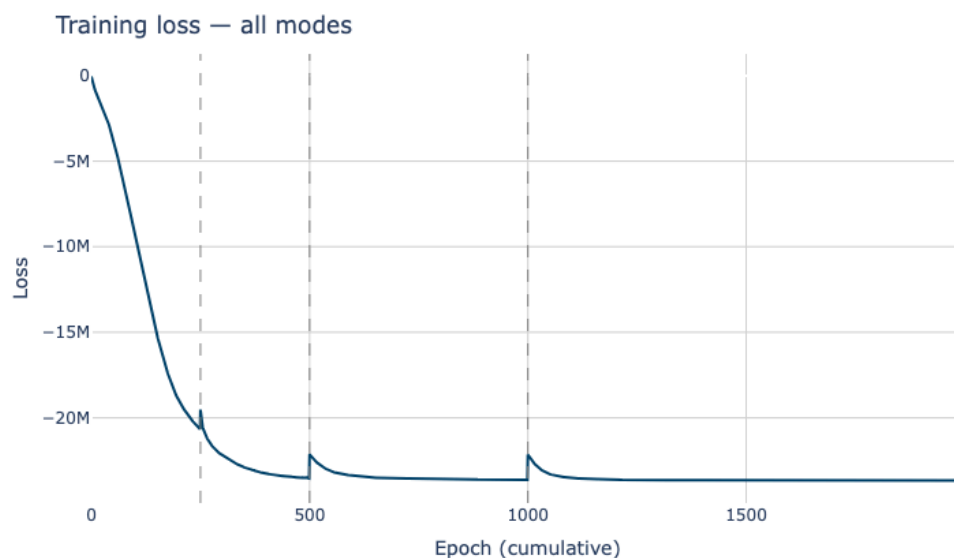
On a quelque chose de mieux que les cas précédents lorsqu'on reste avec peu d'epochs pour chaque mode.

Mais chaque addition de mode renvoie l'énergie à un niveau bien supérieur au précédent donc si on baisse le learning rate, il faut quand même un bon nombre d'epochs pour retrouver une bonne solution après l'ajout d'un mode.

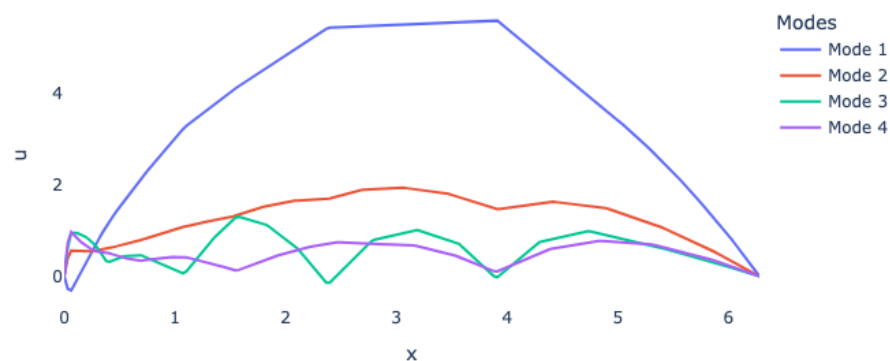
On reproduit fidèlement la dépendance en E .

Mais la partie des basses valeurs n'est reproduite que si on ajoute suffisamment de modes.

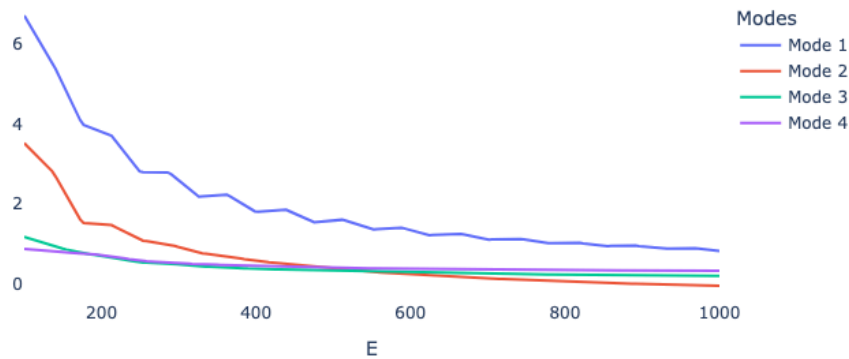
Avec 3 modes, on a du mal sur cette zone critique mais avec 4 on reproduit très bien.



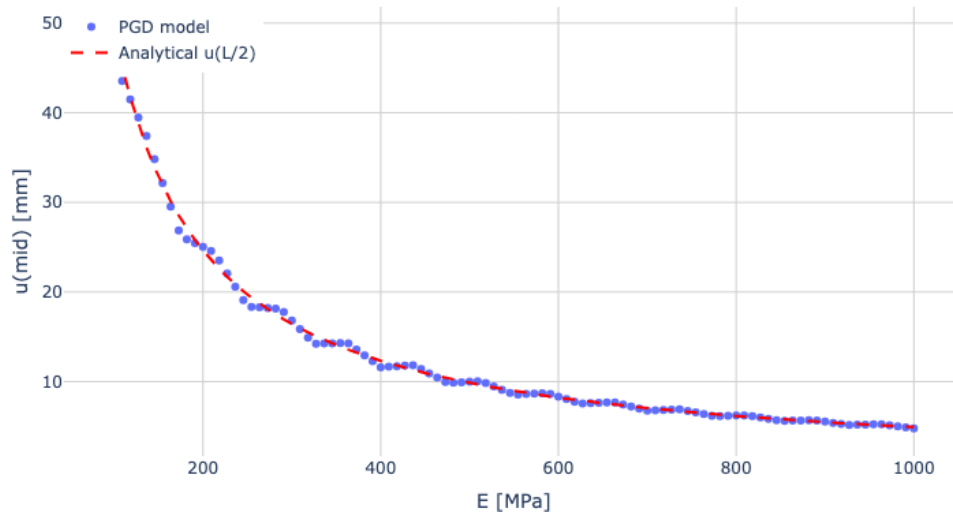
PGD spatial modes



PGD parametric modes



Midpoint deflection $u(x=3.14)$ vs E



Conclusion et discussion

Je n'ai pas implémenté de r-adapt pour le mode paramétrique ce qui aurait pu être intéressant étant donné la difficulté pour le réseau de reproduire la zone des basses valeurs i.e. celle de plus forte variation.

Pas de parallélisation implémentée ici dans le calcul des projections sur une grille commune : nombre d'opération qui évolue en $O(n^2)$ ($n(n+1)/2$ exactement)

L'idée était de créer une history grid, qui contient tous les noeuds des anciens modes.

Et l'on aurait juste à intersecter cette grille avec la grille du nouveau mode.

Mais cela n'est pas adapté si on optimise tous les meshes à la fois par exemple.

On pourrait aussi stocker les anciens modes et leurs dérivées sur une grille assez fine et aller simplement interpoler pour ne pas avoir à différencier de nouveau à chaque fois. Mais encore une fois, si l'on optimise les valeurs nodales de tous les modes à la fois, cela n'est pas possible.

Question MG : Veut-on passer du temps sur le multimillage, qui se généralise difficilement en 2/3D ? Cette façon de gérer la r-adaptivity sans pénalisation est très spécifique au cas d'un domaine 1D.

ADS : Cela pourrait être intéressant pour faire de la r-adaptivity dans les domaines paramétriques d'une PGD car la plupart des paramètres évoluent sur des segments 1D.

Discussion sur la stabilité : est ce qu'il faudrait mettre une taille de maille minimale pour éviter d'avoir des gradients numériques trop grands ? Autre option : régularisation globale sur la qualité du maillage.

TODO :

- Regarder [Lebris et al. 2009], étudier l'optimisation globale vs mode par mode.
Cela revient aux différents algorithmes de [Nouy 2010] PGD-P, PGD-S qui ont différentes propriétés d'optimalité de leur résidus.